3418. Maximum Amount of Money Robot Can Earn

Solved

Medium | Topics | Companies | Hint

You are given an `m x n` grid. A robot starts at the top-left corner of the grid `(0, 0)` and wants to reach the bottom-right corner `(m - 1, n - 1)`. The robot can move either right or down at any point in time.

The grid contains a value `coins[i][j]` in each cell:

- If `coins[i][j] >= 0`, the robot gains that many coins.
- If `coins[i][j] < 0`, the robot encounters a robber, and the robber steals the **absolute** value of `coins[i][j]` coins.

The robot has a special ability to **neutralize robbers** in at most **2 cells** on its path, preventing them from stealing coins in those cells.

Note: The robot's total coins can be negative.

Return the **maximum** profit the robot can gain on the route.

Example 1:

> **Input:** `coins = [[0,1,-1],[1,-2,3],[2,-3,4]]`
>
> **Output:** `8`
>
> **Explanation:**
>
> An optimal path for maximum coins is:
>
> 1. Start at `(0, 0)` with `0` coins (total coins = `0`).
> 2. Move to `(0, 1)`, gaining `1` coin (total coins = `0 + 1 = 1`).
> 3. Move to `(1, 1)`, where there's a robber stealing `2` coins. The robot uses one neutralization here, avoiding the robbery (total coins = `1`).
> 4. Move to `(1, 2)`, gaining `3` coins (total coins = `1 + 3 = 4`).
> 5. Move to `(2, 2)`, gaining `4` coins (total coins = `4 + 4 = 8`).

Example 2:

> **Input:** `coins = [[10,10,10],[10,10,10]]`
>
> **Output:** `40`
>
> **Explanation:**
>
> An optimal path for maximum coins is:
>
> 1. Start at `(0, 0)` with `10` coins (total coins = `10`).
> 2. Move to `(0, 1)`, gaining `10` coins (total coins = `10 + 10 = 20`).
> 3. Move to `(0, 2)`, gaining another `10` coins (total coins = `20 + 10 = 30`).
> 4. Move to `(1, 2)`, gaining the final `10` coins (total coins = `30 + 10 = 40`).

Constraints:

- `m == coins.length`
- `n == coins[i].length`
- `1 <= m, n <= 500`
- `-1000 <= coins[i][j] <= 1000`

Python:

```
class Solution:
    def maximumAmount(self, coins):
        m,n=len(coins),len(coins[0])
        NEG=float('-inf')
        dp=[[NEG]*3 for _ in range(n)]

        for k in range(3):
            dp[0][k]=max(coins[0][0],0) if k>0 else coins[0][0]
```

```
        for j in range(1,n):
            for k in range(2,-1,-1):
                dp[j][k]=max(dp[j][k], dp[j-1][k]+coins[0][j])
                if k>0:
                    dp[j][k]=max(dp[j][k], dp[j-1][k-1])

        for i in range(1,m):
            ndp=[[NEG]*3 for _ in range(n)]
            for j in range(n):
                for k in range(2,-1,-1):
                    if dp[j][k]!=NEG:
                        ndp[j][k]=max(ndp[j][k], dp[j][k]+coins[i][j])
                    if k>0 and dp[j][k-1]!=NEG:
                        ndp[j][k]=max(ndp[j][k], dp[j][k-1])
                    if j>0:
                        if ndp[j-1][k]!=NEG:
                            ndp[j][k]=max(ndp[j][k], ndp[j-1][k]+coins[i][j])
                        if k>0 and ndp[j-1][k-1]!=NEG:
                            ndp[j][k]=max(ndp[j][k], ndp[j-1][k-1])
            dp=ndp

        return dp[n-1][2]
```

JavaScript:

```
/**
 * @param {number[][]} coins
 * @return {number}
 */
var maximumAmount = function(coins) {
    const m = coins.length;
    const n = coins[0].length;
    const NEG = -1e9;

    const dp = Array.from({ length: m }, () =>
        Array.from({ length: n }, () => Array(3).fill(NEG))
    );

    if (coins[0][0] >= 0) {
        dp[0][0][0] = coins[0][0];
    } else {
        dp[0][0][0] = coins[0][0];
        dp[0][0][1] = 0;
    }
```

```
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            for (let k = 0; k <= 2; k++) {
                if (dp[i][j][k] === NEG) continue;

                if (i + 1 < m) {
                    const val = coins[i + 1][j];

                    dp[i + 1][j][k] = Math.max(dp[i + 1][j][k], dp[i][j][k] + val);

                    if (val < 0 && k < 2) {
                        dp[i + 1][j][k + 1] = Math.max(dp[i + 1][j][k + 1], dp[i][j][k]);
                    }
                }

                if (j + 1 < n) {
                    const val = coins[i][j + 1];

                    dp[i][j + 1][k] = Math.max(dp[i][j + 1][k], dp[i][j][k] + val);

                    if (val < 0 && k < 2) {
                        dp[i][j + 1][k + 1] = Math.max(dp[i][j + 1][k + 1], dp[i][j][k]);
                    }
                }
            }
        }
    }

    return Math.max(
        dp[m - 1][n - 1][0],
        dp[m - 1][n - 1][1],
        dp[m - 1][n - 1][2]
    );
};
```

Java:

```
class Solution {
    public int maximumAmount(int[][] coins) {
        int n = coins.length;
        int m = coins[0].length;
        int[][][] dp = new int[n][m][3];

        for (int[][] row : dp) {
            for (int[] col : row) {
```

```
            java.util.Arrays.fill(col, (int)-1e9);
        }
    }

    dp[0][0][1] = 0;
    dp[0][0][2] = 0;
    dp[0][0][0] = coins[0][0];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            for (int k = 0; k < 3; k++) {
                if (i > 0) dp[i][j][k] = Math.max(dp[i][j][k], dp[i - 1][j][k] + coins[i][j]);
                if (i > 0 && k > 0) dp[i][j][k] = Math.max(dp[i][j][k], dp[i - 1][j][k - 1]);
                if (j > 0) dp[i][j][k] = Math.max(dp[i][j][k], dp[i][j - 1][k] + coins[i][j]);
                if (j > 0 && k > 0) dp[i][j][k] = Math.max(dp[i][j][k], dp[i][j - 1][k - 1]);
            }

    return Math.max(dp[n - 1][m - 1][0], Math.max(dp[n - 1][m - 1][1], dp[n - 1][m - 1][2]));
  }
}
```